
Machine Learning for Short-Term Queue Time Prediction at Disneyland's Space Mountain

github.com/jpogue2/csci467-project

Irika Katiyar Jeremy Pogue

Abstract

This project explores short-term queue time prediction for Disneyland's Space Mountain using historical data and machine learning. We engineered features to capture temporal trends, weather conditions, and holiday effects, and evaluated several models including linear regression, Lasso, multilayer perceptron, and XGBoost. The XGBoost model performed best, predicting wait times within 15 minutes of the actual value on the test set. Analysis of high-error cases suggests that ride closures and subtle holiday patterns contributed to inaccuracies. These results reveal the value of context-aware models and demonstrate opportunity for future improvement.

1. Introduction

Disneyland in Anaheim, California is one of the most visited theme parks globally, drawing over 17 million guests in 2023. With its growing popularity, long attraction queues have become a major concern for both visitors and park operators. Since 2014, average queue times have risen by approximately 36% (Koebert, 2025), with some rides exceeding two-hour waits during peak periods. Space Mountain, a flagship attraction in Tomorrowland, often sees queue times between 45 and 90 minutes, especially during peak hours.

1.1. Problem Statement

Accurate short-term¹ queue time prediction is key to managing guest flow, optimizing park operations, and improving the overall visitor experience. With reliable forecasts, guests can make informed decisions about which attractions to visit, helping them make the most of their time in the park.

While traditional queue estimates rely on deterministic methods, machine learning offers a data-driven alternative that in-

corporates historical trends and external factors like weather and special events.

This study investigates the performance of several machine learning models in predicting wait times for Space Mountain using historical data. By refining short-term queue prediction, it aims to support more responsive park operations and improve guest satisfaction.

1.2. Overall Results

We evaluated the impact of feature engineering alongside a range of models, including linear regression, Lasso regression, XGBoost, and a simple multilayer perceptron (MLP). Through model comparison and hyperparameter tuning, we developed a predictor capable of estimating wait times within 15 minutes of the actual values, exceeding our initial expectations. The following sections detail the methods used to achieve this result.

2. Related Work

Every theme park employs proprietary algorithms to perform short-term queue time prediction for its attractions. However, there is no publicly available research applying machine learning to predict time-specific theme park queue times. This section reviews prior work in two related areas: (1) theme park queue time analysis, and (2) queue time prediction in general queuing contexts.

2.1. Theme Park Queue Time Analysis

A 2024 thesis study investigates factors affecting queue times for six Disney World attractions, exploring various prediction models. However, it focuses on daily average wait times rather than time-specific forecasts and relies solely on linear regression and its variants, such as single- and multiple-variable regression. The study concludes that seasonality has the greatest impact, with multiple linear regression performing best (Ferman, 2024).

Similarly, a 2023 study examines the impact of Fast Pass systems on attraction wait times, aiming to optimize pric-

¹In this paper, short-term queue time prediction refers to forecasts up to 30 minutes into the future.

ing and allocation. Instead of using machine learning, it employs a mathematical model based on factors such as queuing methods, guest purchasing behavior, and ride capacity (Li & Li, 2023).

While both studies are related, neither incorporates temporality as a predictive feature. This gap motivates our focus on time-specific queue prediction using machine learning techniques.

2.2. General Queue Time Prediction

Several studies have applied machine learning to wait time prediction in healthcare. A 2018 study evaluated neural networks, support vector machines, K-nearest neighbors, and random forests for predicting radiology wait times. It used variables such as patient arrival time, exam timing, and exam type, and found that model performance varied by examination type using RMSE as the evaluation metric (Curtis et al., 2018).

A 2021 study focused on deep learning for emergency room wait time prediction. It compared optimization algorithms for training neural networks—SGD, Adam, RMSprop, and AdaGrad—using features like arrival time, queue length, staff availability, and patient data. SGD yielded the lowest prediction error (Hijry & Olawoyin).

These studies demonstrate the effectiveness of machine learning in wait time prediction across domains, but none have been applied specifically to theme park attractions, a gap this study aims to address.

3. Dataset and Evaluation

3.1. Preliminary Dataset

We constructed a preliminary dataset containing features that may influence queue times for Disneyland’s Space Mountain. The dataset includes the following columns: "Wait Time", "Date", "Time", "Day of Week", "Month", and "Time of Day". It contains 79,655 examples.

- **Wait Time:** Historical queue time data at 5-minute intervals is available for most global theme park attractions on queue-times.com (Bull, 2014). A Python web scraping script was used to collect data specifically for Disneyland’s Space Mountain by sending GET requests to: <https://queue-times.com/en-US/parks/16/rides/284>, and the JSON response was parsed to obtain the date, time, and queue time data. The retrieved data includes queue times with 5-minute precision.
- **Date:** The dataset covers December 6, 2014 to April 2, 2025. Data is unavailable between March 13, 2020 and April 30, 2021 due to the park’s COVID-19 closure.

Dates are formatted as "DD/MM/YY".

- **Time:** The original data ranges from 00:00 AM to 11:55 PM. To reduce training time while maintaining responsiveness, we only sampled every 30 minutes and excluded times between 00:00 AM and 7:00 AM (inclusive), as the park is rarely open during these hours. Times are formatted as "HH:MM:SS". Note: Prior to COVID-19, queue data was logged on the hour (e.g., "10:00"). After reopening, timestamps shifted to one minute past the hour (e.g., "10:01"), so one minute was subtracted from all post-2021 times for consistency.
- **Day of Week:** Derived from the "Date" column by converting to a datetime object and extracting the weekday name (e.g., "Monday").
- **Month:** Extracted directly from the "Date" column as an integer from 1 to 12.
- **Time of Day:** Converted from the "Time" column into an integer representing minutes after midnight, ranging from 450 (7:30 AM) to 1410 (11:30 PM).

To train and test these models, we split the dataset by year: data from 2014–2019 and 2022 is used for training (50962 examples), data from 2023 is used for the development set (8215 examples), and data from 2024–2025 is reserved for testing (11288 examples). We exclude 2020 and 2021 due to anomalous behavior caused by COVID-19. This results in an approximate 72.3%/11.7%/16.0% train-dev-test split.

3.2. Final Dataset

The original dataset features remained unchanged since the midterm report. However, based on insights from the manual error analysis conducted during the midterm, several new features were added, including holiday indicators, weather conditions, and prior wait times. These additions are described in more detail in the Methods section.

3.3. Evaluation

Predicting queue times is a regression task, so we use Root Mean Squared Error (RMSE) as our evaluation metric. RMSE quantifies the average magnitude of prediction errors and penalizes larger deviations more heavily, making it suitable for applications where high precision is desirable, such as queue forecasting for real-time decision-making.

4. Methods

4.1. Baseline Statistical Model

We implemented a statistical model as a baseline to compare against future machine learning approaches. This model cal-

culates the average wait time for each 30-minute interval across all days of the week, aggregated over all available months and years. The result is a dictionary mapping each (day of week, time of day) pair to its corresponding average wait time. During evaluation, predictions are made by looking up the average wait time for the given day and time. We selected this baseline based on the assumption that day of the week and time of day are the most influential factors affecting queue times.

4.2. Linear Regression Model

As an initial machine learning approach, we implemented a basic linear regression model to compare against our statistical baseline due to its simplicity and interpretability. This required additional pre-processing of our dataset.

First, we applied one-hot encoding to the "Day of Week" feature, creating a separate binary indicator for each day. This is a standard technique for handling categorical variables in machine learning, which cannot natively interpret non-numeric data.

Next, we engineered three additional features to better capture observed patterns in the training data (see Figure 1). Specifically, we noticed: (1) a nonlinear trend in queue times throughout the day, where wait times peak around midday and drop off in the morning and evening; and (2) a cyclical pattern across the week.

To model the first pattern, we included a quadratic feature, $(\text{Time of Day})^2$, allowing the regression to approximate the bell-shaped curve of daily queue behavior. To model the second pattern, we added $\sin(\text{Time of Day})$ and $\cos(\text{Time of Day})$ features, which help represent the cyclical nature of time over a day and across the week.

All of the above features: Time of Day, $(\text{Time of Day})^2$, $\sin(\text{Time of Day})$, $\cos(\text{Time of Day})$, and the seven indicator features for Monday through Sunday were passed into an sklearn Linear Regression model. We used sklearn's closed-form LinearRegression with default settings, as no regularization or tuning was applied.

4.3. Dataset Enhancements

To improve prediction accuracy, we expanded the dataset with additional features that reflect contextual and temporal factors likely to influence wait times.

4.3.1. CONTEXTUAL FEATURES

We first introduced features related to external conditions that might affect park attendance and ride popularity:

- **Is Holiday:** Indicates whether the date is a recognized

U.S. holiday. This was determined using the Holidays Python package, which provides holiday data for 171 countries, including the U.S. (P., 2023).

- **Is Holiday Adjacent:** Flags whether the date is the day before or after a holiday. These adjacent days often see increased crowd levels due to extended weekends or travel patterns.
- **Temperature:** Historical hourly temperature data retrieved via the Open Meteo API (Zippenfenig, 2023), based on the park's geographic coordinates: (33.8121, -117.9190).
- **Precipitation:** Hourly precipitation data, also obtained from Open Meteo, capturing conditions that may affect guests' willingness to queue outdoors.

To handle gaps in the weather data, we applied forward and backward filling within each day. Rows with missing weather values after this step were removed to maintain data integrity.

These features were added based on their potential to influence guest behavior; weather conditions and holidays often affect whether people choose to visit the park or wait in outdoor queues.

4.3.2. TEMPORAL FEATURES

To better model the time-dependent nature of queue dynamics, we added six features designed to capture patterns from prior time intervals:

- **Lag 1-3:** Wait times at the same time of day on the previous 1, 2, and 3 days, respectively. These features help capture recurring temporal patterns or trends in guest flow.
- **Rolling Mean Lag 3:** The average of the Lag 1-3 values, intended to smooth out short-term fluctuations and highlight broader trends.
- **Wait Time 30 Minutes Ago:** The most recent available wait time from 30 minutes earlier. If unavailable (e.g., early in the day), this is imputed using the Lag 1 value. Assuming the use case of short-term prediction, this value will always be available as a feature during testing.
- **Wait Time 1 Hour Ago:** Similar to the 30-minute feature but taken from one hour prior. If this value is missing, it is imputed using the Rolling Mean Lag 3 value.

These additions were motivated by the assumption that short-term historical trends, along with cyclical daily behavior,

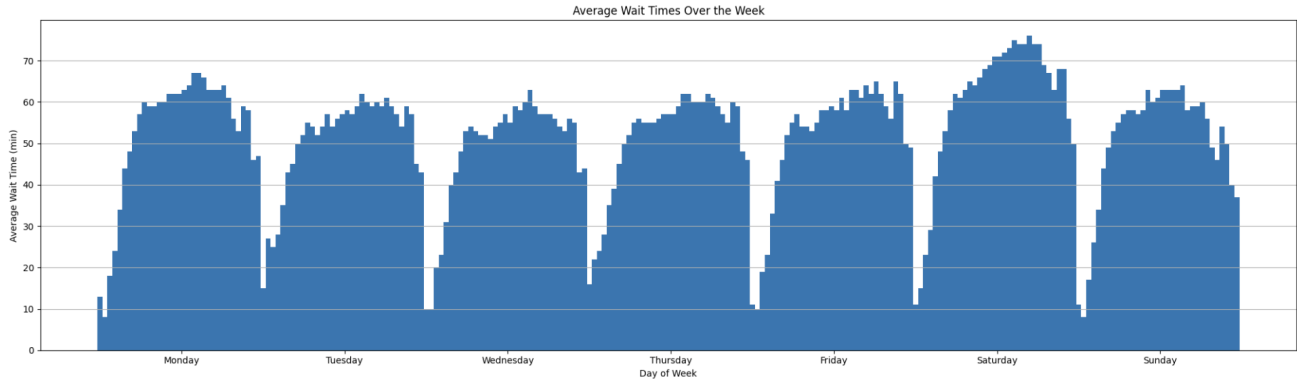


Figure 1. Average queue time over the course of a full week, showing the cyclical trend across days and time of day.

could help machine learning models better capture underlying queue patterns.

4.4. Model Comparison

After enhancing the dataset with contextual and temporal features, we conducted a comprehensive comparison of machine learning models to evaluate their effectiveness in short-term queue time prediction. Our goal was to systematically map input features (such as holiday indicators, weather conditions, and prior wait times) to predicted wait times, and determine which models best captured the underlying patterns in the data.

We began with a baseline Linear Regression model, implemented using scikit-learn’s `LinearRegression` with default settings. This closed-form solution served as a reference point, as it assumes a linear relationship between features and target values without regularization.

To improve generalization and reduce the risk of overfitting, we introduced Lasso Regression (linear regression with L1 regularization), which penalizes large coefficients proportional to their absolute sum. We tuned the regularization strength using a grid of alpha values: [0.01, 0.1, 1.0, 10.0, 100.0].

Next, we explored XGBoost, a powerful ensemble method based on gradient-boosted decision trees. This model is well-suited for capturing complex, non-linear interactions and handling heterogeneous feature types. We used the `XGBRegressor` from the `xgboost` library and performed a grid search over the following hyperparameters:

- `n_estimators` = [100, 200]: Number of boosting rounds (trees)
- `learning_rate` = [0.01, 0.1, 0.2]: Step size shrinkage to prevent overfitting

- `max_depth` = [3, 5, 7]: Controls the complexity of individual trees

Finally, we evaluated a Multilayer Perceptron (MLP) using scikit-learn’s `MLPRegressor`. This neural network model introduces non-linearity and is capable of learning abstract representations of the input. To explore the design space, we performed an extensive grid search across 1,080 hyperparameter combinations:

- `hidden_layers` = [(32,), (64,), (128,), (256,), (64, 32), (128, 64), (128, 128), (256, 128), (256, 256)]: Controls the size and quantity of hidden layers in the model architecture
- `activations` = ['relu', 'tanh']: Non-linearity functions
- `learning_rates` = [1e-4, 5e-4, 1e-3, 5e-3, 1e-2]: Step size shrinkage to prevent overfitting
- `alphas` = [0.0001, 0.001, 0.01, 0.1]: L2 regularization term
- `batch_sizes` = [32, 64, 128]: Number of samples used per iteration of training

This systematic approach allowed us to evaluate a wide range of architectures and training configurations.

In contrast to our midterm report, which relied on fewer features and simpler models, this final approach incorporates temporal context, environmental conditions, and a broader search over model types and hyperparameters. This expanded methodology allowed us to capture more nuanced patterns in the data and significantly improve prediction accuracy.

In the following Experiments section, we will analyze the results of this comparison and show that the XGBoost model outperformed all others.

5. Experiments

We began our experimentation by comparing the performance of three initial models: a statistical baseline, linear regression using the original dataset, and linear regression using the enhanced dataset. All models were evaluated using the same train-dev-test split described in the Dataset section. The results, shown in Figure 2, demonstrate a significant reduction in prediction error when using the enhanced dataset. This confirms that incorporating features such as holidays, weather, and temporal lag improves the model’s ability to capture relevant patterns in queue behavior.

Given this improvement, all subsequent models were trained and evaluated using the enhanced dataset exclusively.

A summary of final model performance, measured by Root Mean Square Error (RMSE) on the test set, is presented in Table 1. Where applicable, the hyperparameters used are included.

5.1. Linear and Regularized Models

Linear regression with the enhanced dataset showed noticeable improvement over both the baseline and the original dataset version. Adding L1 regularization via Lasso further reduced test error. The best Lasso model used $\alpha = 0.01$, selected based on development set performance (Dev RMSE = 17.57). Regularization likely helped by mitigating overfitting and addressing collinearity among features.

5.2. Neural Network

Our MLP model achieved strong performance, particularly after an extensive hyperparameter search. We evaluated over 1,000 combinations, varying layer architecture, activation function, learning rate, regularization strength, and batch size. The best-performing configuration is listed in Table 1, with the top 10 combinations and corresponding development RMSEs ranked in Table 2.

While the MLP was capable of modeling non-linear relationships, it required careful tuning and was more sensitive to hyperparameters and training stability compared to tree-based methods.

5.3. Tree-Based Model: XGBoost

XGBoost achieved the best overall performance, predicting wait times within 15 minutes of the actual value. This model was tuned using a grid search over key hyperparameters: number of estimators, learning rate, and maximum depth. The top 10 configurations, along with their development RMSEs, are shown in Table 3.

The superior performance of XGBoost can be attributed to its ability to model non-linear relationships and feature inter-

actions effectively. Additionally, its robustness to collinearity and capacity to handle diverse feature types made it particularly well-suited for our dataset.

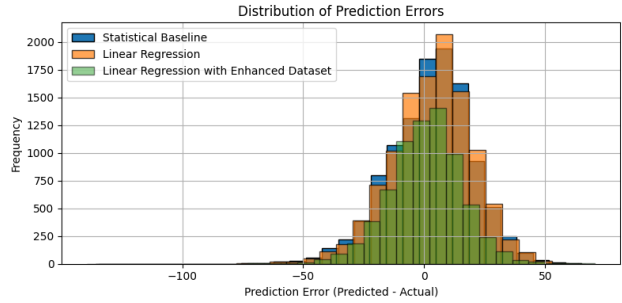


Figure 2. Distribution of prediction errors for the statistical baseline, basic linear regression, and linear regression using the enhanced dataset. Most errors fall within ± 25 minutes, with reduced variance in the enhanced model.

6. Discussion

To better evaluate model performance, we randomly sampled 100 instances from the test set and generated predictions using the best-performing model, XGBoost. These samples were ranked by absolute prediction error, and the top 10 are presented in Table 4, alongside errors from the statistical baseline model.

Manual inspection of these high-error cases revealed several consistent patterns. While the enhanced dataset addressed many of the original model’s shortcomings, such as lacking holiday and weather data, it still omits a critical factor: ride closures. No reliable method was found to scrape or retrieve this information at scale.

Ride closures plausibly explain the largest errors in four examples (Rows 1, 3, 6, and 9), all of which involved the ride being closed for at least an hour earlier in the day. Such interruptions can lead to irregular guest behavior or atypical queue dynamics, especially if operators are delayed in resuming standard operations.

Another notable case is December 23, 2024 (Row 2). In 2024, Christmas fell on December 25, so December 23 is neither a holiday nor a holiday-adjacent date in our feature set, although it still experienced holiday guest traffic. Therefore, the model underpredicted the wait time by 33.39 minutes. A more nuanced holiday encoding that completely accounts for proximity to major holidays could have better captured this seasonal surge.

Additionally, the statistical baseline model outperformed XGBoost in several of these worst-case scenarios. This suggests that XGBoost may overfit to short-term lag fea-

Table 1. Test RMSE and selected hyperparameters for each model tested. All models except the statistical baseline and basic linear regression were trained on the enhanced dataset. XGBoost achieved the lowest RMSE.

MODEL	TEST RMSE	SELECTED HYPERPARAMETERS
STATISTICAL BASELINE	17.32	—
BASIC LINEAR REGRESSION	17.25	DEFAULT
LINEAR REGRESSION (ENHANCED)	15.10	DEFAULT
LASSO REGRESSION	15.02	$\alpha = 0.01$
XGBOOST REGRESSOR	14.62	$n_est = 200, depth = 5, \eta = 0.1$
MLP	15.44	(256, 256), RELU, $lr = 1E-3, \alpha = 0.001, batch = 32$

Table 2. Top 10 MLP hyperparameter combinations ranked by development set RMSE. The configuration used for final testing is bolded.

HIDDEN_LAYERS	ACTIVATION	LEARNING_RATE	ALPHA	BATCH_SIZE	DEV RMSE
(256, 256)	RELU	0.0010	0.0010	32	16.0720
(256, 256)	RELU	0.0005	0.1000	32	16.1030
(256, 256)	RELU	0.0005	0.0001	32	16.1083
(256, 256)	RELU	0.0005	0.0010	32	16.1142
(256, 256)	RELU	0.0005	0.0100	32	16.1166
(128, 64)	RELU	0.0100	0.0100	32	16.1461
(256, 256)	TANH	0.0050	0.0010	128	16.2021
(128, 64)	RELU	0.0050	0.0010	32	16.2131
(256, 256)	TANH	0.0100	0.0010	128	16.2229
(64,)	TANH	0.0050	0.0100	32	16.2372

tures, making it vulnerable to sudden changes not reflected in recent history.

These findings indicate that the model could benefit from two key enhancements:

1. Incorporating ride closure data, either through third-party API or operational logs.
2. Expanding the holiday feature set to include multi-day holiday effects.

Together, these improvements could enhance the model’s robustness and reduce vulnerability to anomalous crowd patterns.

7. Conclusion

This project explored short-term queue time prediction using a range of machine learning models and engineered features. We found that incorporating temporal, weather, and holiday data significantly improved performance, with the XGBoost Regressor achieving predictions within 15 minutes of actual wait times. This result far exceeded our initial expectations.

In comparing models, we saw that tree-based methods like XGBoost effectively captured nonlinear patterns in time-dependent data, outperforming both linear models and neural networks. The project also revealed opportunities for further improvement, particularly in handling ride closures and refining holiday features.

Overall, we gained valuable insights into both the predictive modeling process and the unique challenges of time-sensitive real-world data.

References

- Bull, Z. Parks - space mountain, 2014. URL <https://queue-times.com/en-US/parks/16/rides/284>.
- Curtis, C., Liu, C., Bollerman, T. J., and Panykh, O. S. Machine learning for predicting patient wait times and appointment delays. *Journal of the American College of Radiology*, 15(9):1310–1316, 2018. ISSN 1546-1440. doi: <https://doi.org/10.1016/j.jacr.2017.08.021>. URL <https://www.sciencedirect.com/science/article/pii/S1546144017310141>.
- Ferman, D., 2024. URL <https://scholarworks.calstate.edu/downloads/td96kc050>.
- Hijry, H. and Olawoyin, R. Predicting patient waiting time in the queue system using ... URL <https://www.ieomsociety.org/journals/volume3/vol-3-no-1-3.pdf>.
- Koebert, J. Want to avoid lines at disney? data reveals how to get the most out of your next trip to the parks, January 16 2025. URL <https://financebuzz.com/disney-wait-times-study>.

Table 3. Top 10 XGBoost hyperparameter combinations based on development set RMSE. The best-performing configuration is bolded.

N_ESTIMATORS	MAX_DEPTH	LEARNING_RATE	DEV RMSE
200	5	0.10	15.40
100	5	0.10	15.40
200	3	0.10	15.41
200	3	0.20	15.43
100	3	0.10	15.44
100	5	0.20	15.47
100	3	0.20	15.48
100	7	0.10	15.50
200	7	0.01	15.54
200	5	0.20	15.56

Table 4. Top 10 largest absolute prediction errors from the XGBoost model on random sample of test set, compared against the statistical baseline. Positive errors indicate overestimation, and negative errors indicate underestimation.

INDEX	DATE	TIME	DAY	Y-ACTUAL	Y-PRED (XG-BOOST)	ERROR (XG-BOOST)	ERROR (STAT)
1	2024-12-09	14:30	MON.	80.0	46.46	-33.54	-18
2	2024-12-23	18:30	MON.	85.0	51.61	-33.39	-22
3	2024-10-02	10:30	WED.	15.0	43.81	28.81	28
4	2024-02-27	08:30	TUES.	10.0	35.28	25.28	15
5	2024-05-28	20:30	TUES.	60.0	85.05	25.05	-1
6	2024-01-04	16:30	THURS.	65.0	89.94	24.94	-6
7	2024-05-04	10:00	SAT.	70.0	47.76	-22.24	-22
8	2024-01-25	18:00	THURS.	80.0	57.77	-22.23	-20
9	2024-11-08	17:00	FRI.	30.0	52.15	22.15	28
10	2024-03-19	11:00	TUES.	35.0	57.14	22.14	15

Li, J. and Li, Q. Analysis of queue management in theme parks introducing the fast pass system, Jul 2023. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC10362101/#abs0010>.

P., V. Python holidays library. <https://pypi.org/project/holidays/>, 2023. Version 0.25.

Zippenfenig, P. Open-meteo.com weather api, 2023. URL <https://open-meteo.com/>.